Pygame: Qué es, para qué se utiliza y su relación con ciclos, eventos, estructuras de datos y funciones

Resumen ejecutivo

Pygame es una biblioteca libre y de código abierto, multiplataforma, para desarrollar aplicaciones multimedia y videojuegos en 2D con Python. No es un motor de juego con editor: es un conjunto de módulos construido sobre SDL (Simple DirectMedia Layer) que abstrae las funciones más comunes —ventana, dibujo, imágenes, entrada, sonido y tiempo— para que la escritura de estos programas sea más intuitiva 1. Su valor pedagógico central consiste en que el programa del estudiante *es* el juego: el bucle, el dibujo, las colisiones y la disposición del nivel se escriben en código 2. Por eso, los temas de clase —ciclos, eventos, estructuras de datos y funciones— no son un prerrequisito abstracto, sino la arquitectura misma de un programa Pygame. La expresión "tomar mucho o poco de la librería" se refiere a su modularidad y a su bajo nivel de imposición: es posible usar solamente pygame.display y pygame.draw para un experimento visual, o bien incorporar pygame.sprite, pygame.mixer, pygame.font y pygame.time para un juego completo, e incluso inicializar selectivamente solo los submódulos necesarios.

1. Qué es Pygame

Pygame is a free and open-source cross-platform library for the development of multimedia applications like video games using Python. It uses the Simple DirectMedia Layer library and several other popular libraries to abstract the most common functions, making writing these programs a more intuitive task. 1

La definición oficial contiene tres ideas que conviene separar:

1. **Biblioteca, no motor.** Pygame no trae editor de escenas, editor de niveles, física integrada ni sistema de animación por defecto. Ofrece piezas: abrir una ventana, dibujar sobre superficies, leer teclado y ratón, reproducir sonido y controlar el tiempo. El resto —reglas, niveles, colisiones, puntuación, estados— lo escribe quien programa.
2. **Envoltorio de SDL.** Pygame es un *wrapper* alrededor de SDL 3. SDL es una biblioteca en C que usan muchos juegos comerciales para ventana, gráficos, entrada y audio; al envolverla en Python, Pygame permite desarrollo en tiempo real sin la mecánica de bajo nivel de C 4. Esa es la razón por la que el mismo código corre en Windows, macOS y Linux.
3. **Multimedia más allá de juegos.** Aunque su uso más conocido es el videojuego 2D, la descripción oficial habla de "aplicaciones multimedia", lo que incluye visualizaciones, simulaciones, juguetes de física, autómatas celulares y experimentos visuales 5.

Existe además una distinción de versiones relevante para clase: el Pygame original y **pygame-ce** (Community Edition), una bifurcación creada en 2023 por la mayoría de los desarrolladores activos. Ambas se importan con import pygame y casi todo el código corre en ambas; pygame-ce se actualiza con mayor frecuencia 5. Para un curso, lo práctico es instalar una de las dos y no mezclarlas.

1.1 Historia breve: de PySDL a pygame-ce

Pygame nació en el verano de 2000, cuando Pete Shinners quiso unir Python con SDL y crear algo que aprovechara de verdad Python, en lugar de obligar a escribir estilo C [13]. Su objetivo declarado fue que fuera fácil hacer las cosas simples y directo hacer las difíciles [13]. El proyecto arrancó en octubre de 2000 y seis meses después salió la versión 1.0 [13]. Con el tiempo se convirtió en un proyecto comunitario, publicado bajo licencia LGPL, y más tarde apareció pygame-ce como bifurcación de los antiguos desarrolladores principales [14].

Esa historia explica dos rasgos que todavía se sienten al usarlo. Primero, su filosofía es práctica y minimalista: no intenta ser un entorno completo, sino una caja de herramientas. El propio Shinners advierte que Pygame es bastante de bajo nivel para escribir juegos y que quien programa pronto necesitará envolver funciones comunes en su propio entorno, con la ventaja de que no hay nada dentro de Pygame que se interponga [13]. Segundo, su longevidad explica la cantidad de tutoriales, ejemplos y respuestas disponibles: es una de las puertas de entrada más documentadas a la programación de juegos en Python.

1.2 Instalación y primera verificación

La instalación recomendada hoy es con pip, el instalador de paquetes de Python, eligiendo una sola de las dos variantes [15]:

Windowspy -m pip install pygame-ce# macOS y Linuxpython3 -m pip install pygame-ce

Escribir python3 -m pip en lugar de pip a secas importa, porque instala Pygame para exactamente el Python que después ejecutará el juego [15]. Nunca deben instalarse pygame y pygame-ce a la vez, porque comparten el nombre pygame y se sobrescriben entre sí [15]. Para un proyecto real conviene usar un entorno virtual, de modo que los paquetes no se mezclen con otros proyectos [15]:

python3 -m venv .venvsource .venv/bin/activate # Windows: .venv\Scripts\activatepython -m pip install pygame-ce

La verificación es inmediata:

python3 -c "import pygame; print(pygame.version.ver)"python3 -m pygame.examples.aliens

Si se abre una ventana con un juego de ejemplo, la instalación funciona. El error más común en este paso, ModuleNotFoundError: No module named 'pygame', casi siempre significa que Pygame se instaló para un Python distinto del que ejecuta el script; la solución es instalarlo con ese mismo Python y seleccionar ese intérprete en el editor [15].

2. Para qué se utiliza

Uso	Qué hace Pygame ahí	Ejemplo de clase
Aprender a hacer juegos	Sin editor ni motor oculto: el bucle, el dibujo y las colisiones son código legible	Primer programa: ventana y cuadrado que se mueve con flechas

Uso	Qué hace Pygame ahí	Ejemplo de clase
Juegos 2D de cualquier estilo	Arcade, plataformas, disparos, puzles, cartas y juegos por baldosas	Clon de Pong, Snake, Dodger, rompe-bloques
Simulaciones y experimentos visuales	Partículas, juguetes de física, autómatas celulares, visualización de ordenamiento	Simulación de rebote, visualizador de burbuja
Game jams y prototipos	Programas pequeños, que arrancan en un segundo y se comparten como archivos Python	Prototipo de una idea en una sesión

Sus límites también son instructivos: es una opción débil para juegos 3D, para juegos que necesitan un editor para disponer niveles, o para publicar en teléfonos y consolas [5]. Esa limitación no es un defecto pedagógico; es precisamente lo que obliga a entender cómo funciona un juego por dentro.

3. Arquitectura por módulos

Pygame es un conjunto de módulos, cada uno con un trabajo [5]:

Módulo	Responsabilidad
pygame.display	Controla la ventana y la pantalla
pygame.draw	Dibuja rectángulos, círculos, líneas y polígonos
pygame.image	Carga imágenes PNG y JPG
pygame.Rect	Guarda posición y tamaño, mueve cosas y verifica colisiones
pygame.event, pygame.key, pygame.mouse	Lee teclado y ratón
pygame.sprite	Agrupa objetos de juego y verifica colisiones entre ellos

Módulo	Responsabilidad
pygame.font	Convierte texto en una imagen dibujable
pygame.mixer	Reproduce efectos de sonido y música
pygame.time	Mantiene una tasa de cuadros estable y ejecuta temporizadore
pygame.transform	Escala, rota y voltea superficies
pygame.surface / Surface	Áreas de píxeles de colores sobre las que se dibuja

Dos tipos merecen atención especial porque aparecen en casi todo programa: Surface y Rect. Una Surface es un área de píxeles; la superficie devuelta por pygame.display.set_mode() es especial porque lo que se dibuja en ella se muestra en pantalla [6]. Un Rect representa la ubicación y el tamaño de un espacio rectangular, con atributos como topleft, width y height, y con un método colliderect() para verificar si se intersecta con otro Rect [6].

3.1 El sistema de coordenadas: el primer detalle que confunde

Pygame usa un sistema de coordenadas con origen (0, 0) en la esquina superior izquierda de la ventana: el eje X crece hacia la derecha y el eje Y crece hacia abajo [16]. Eso es distinto del sistema matemático habitual, donde Y crece hacia arriba, y es la causa de muchos errores de principiantes: un objeto que debería subir baja, o un texto aparece fuera de la ventana.

Las consecuencias prácticas son concretas:

- Para mover algo hacia arriba hay que **restar** a y; para bajar, **sumar** a y.
- El centro de una ventana de 640 x 480 es (320, 240), no (0, 0).
- Un Rect con top = -50 está fuera de la pantalla por arriba; uno con left > ancho ya salió por la derecha.
- El ratón devuelve su posición en ese mismo sistema, por lo que event.pos se puede comparar directamente con rectángulos del juego.

Entender este sistema es previo a cualquier movimiento, rebote o colisión, porque todas esas operaciones son aritmética sobre coordenadas.

3.2 Superficies, dibujo y doble búfer

Una Surface es un área de píxeles de colores. La superficie que devuelve pygame.display.set_mode() representa los gráficos mostrados; cualquier dibujo sobre ella se

vuelve visible en el monitor [13]. Pygame admite varios formatos de imagen a través de SDL_image, incluidos BMP, JPG, PNG, TGA y GIF [13].

Dos operaciones son el corazón del dibujo:

- **blit()**: copiar los colores de píxeles de una imagen a otra. Se le pasa una Surface origen y una posición, que puede ser un Rect o un par de números [13]. Dibujar un personaje es, en el fondo, hacer blit de su imagen sobre la pantalla en la posición de su Rect.
- **flip() / update()**: Pygame maneja la pantalla con doble búfer. Cuando se termina de dibujar, pygame.display.flip() hace visible todo lo dibujado; ese búfer garantiza que solo se vean cuadros completamente dibujados, sin mitades a medio crear [13].

De ahí sale una regla de oro para depurar: si algo no aparece, casi siempre falta flip(), se dibujó antes del fill() y quedó tapado, o las coordenadas están fuera de la ventana. El orden correcto dentro de cada cuadro es: limpiar con fill(), dibujar todo, y al final flip() una sola vez.

El módulo pygame.draw complementa el blit con formas: rect, circle, line, polygon, ellipse y versiones antialiasing como aalines. No hay una función dedicada de triángulo: se dibuja con polygon pasándole tres puntos. El parámetro width = 0 rellena la forma; un valor mayor dibuja solo el contorno. Estas funciones son ideales para prototipos sin imágenes: un juego completo puede hacerse solo con rectángulos y círculos de colores.

3.3 Sprites y grupos: de objetos sueltos a objetos organizados

Cuando un juego tiene más de unas pocas cosas en movimiento, cada una necesita su imagen, su posición y su comportamiento juntos. Para eso existen Sprite y Group [17]. Un sprite es una clase que hereda de Sprite y define dos atributos, image y rect [17]. Las tres reglas son: llamar primero a super().__init__(), asignar self.image a una Surface y asignar self.rect a su posición, normalmente con self.image.get_rect(...) [17]. El comportamiento del sprite se pone en update(); Group.update() pasa los argumentos que reciba, de modo que group.update(dt) entrega el delta time a cada sprite [17].

Método de grupo	Qué hace
group.add(*sprites)	Agrega sprites
group.update(*args)	Llama a update(*args) en cada sprite
group.draw(surface)	Dibuja la image de cada sprite en su rect
group.remove(*sprites)	Quita sprites de ese grupo

Método de grupo	Qué hace
group.empty()	Quita todos los sprites
sprite.kill()	Quita el sprite de todos los grupos a los que pertenece
sprite.alive()	True si está en al menos un grupo

Un sprite puede estar en varios grupos a la vez. Una disposición común es un grupo con todo, para dibujar, más grupos por rol, enemigos, balas o monedas, para colisiones [17]. Existen variantes como GroupSingle, que guarda como máximo un sprite, por ejemplo el jugador, y LayeredUpdates, que dibuja por capas para controlar qué queda delante [17]. Un detalle de rendimiento importante: cargar la imagen una sola vez y compartirla entre muchos sprites del mismo tipo; cargar el archivo en cada __init__ funciona para pocos sprites, pero ralentiza un juego que genera cientos [17].

Ejemplo mínimo de sprite:

class Moneda(pygame.sprite.Sprite): def __init__(self, x, y, imagen): super().__init__() self.image = imagen self.rect = self.image.get_rect(topleft=(x, y)) self.velocidad = 2 def update(self): self.rect.y += self.velocidad if self.rect.top > 480: self.kill()monedas = pygame.sprite.Group()monedas.add(Moneda(100, 0, imagen_moneda))# En el bucle:monedas.update()monedas.draw(pantalla)

Aquí se ve la ventaja frente a la lista de diccionarios: el bucle ya no maneja monedas una a una; el grupo las actualiza, las dibuja y las elimina cuando salen de pantalla.

4. Relación con los temas vistos en clase

4.1 Ciclos: el game loop como ciclo while

La estructura de los juegos considerados en la literatura introductoria sigue siempre el mismo patrón fijo: sección de configuración, luego entrada al bucle de juego [7]. El bucle hace continuamente cuatro cosas: sondea eventos, actualiza estructuras de datos u objetos, dibuja el estado actual en una superficie no visible y pone esa superficie en pantalla [7].

import pygamepygame.init()pantalla = pygame.display.set_mode((480, 480))reloj = pygame.time.Clock()corriendo = Truewhile corriendo: # 1. Eventos for evento in pygame.event.get(): if evento.type == pygame.QUIT: corriendo = False # 2. Actualizar estado # 3. Dibujar pantalla.fill((0, 200, 255)) pygame.display.flip() # 4. Controlar el tiempo reloj.tick(60)pygame.quit()

Este es el punto donde el ciclo while deja de ser un ejercicio abstracto. La condición corriendo es el interruptor del juego: mientras sea True, el juego vive; cuando el usuario cierra la ventana, se vuelve False y el bucle termina. El tutorial oficial presenta exactamente ese patrón: introducir una

variable booleana running, verificar el evento QUIT dentro del bucle y llamar a pygame.quit() al salir [3].

Dentro del bucle principal aparecen además los otros tipos de ciclo de clase:

- **for sobre la cola de eventos.** for event in pygame.event.get() recorre la lista de eventos ocurridos desde la última revisión.
- **for sobre colecciones de juego.** Por ejemplo, for b in baddies: para mover y dibujar cada enemigo.
- **Ciclos anidados.** Para dibujar un tablero, el bucle exterior recorre filas y el interior recorre columnas; en cada iteración se calcula el rectángulo del cuadro multiplicando fila y columna por el tamaño del cuadro [7].
- **while True con break.** Algunos programas usan un bucle infinito con salida explícita cuando se detecta QUIT, lo que permite discutir cuándo conviene una bandera y cuándo conviene break.

El control de velocidad introduce un concepto adicional: el tiempo. clock.tick(60) hace dos cosas: limita la tasa de cuadros —si el cuadro terminó temprano, espera, de modo que el bucle corre como máximo 60 veces por segundo— y devuelve el tiempo desde el último tick, en milisegundos [8]. Sin esa llamada, un juego simple correría cientos de cuadros por segundo: todo se movería demasiado rápido y el procesador trabajaría al máximo [8]. El paso siguiente, el *delta time* (dt = clock.tick(60) / 1000), permite mover por velocidad por tiempo en lugar de por píxeles por cuadro, de modo que la velocidad no dependa de la tasa de cuadros [8].

Concepto de ciclo	Dónde aparece en Pygame
while con bandera	Bucle principal del juego (while corriendo:)
for sobre lista	Cola de eventos, lista de enemigos, balas, partículas
Ciclos anidados	Tableros, rejillas, mapas por baldosas
break / salida	Cierre de ventana, fin de partida, cambio de estado
Temporización	Clock.tick(60), get_fps(), delta time

4.2 Eventos: entrada como datos, no como interrupciones mágicas

En el tutorial oficial, el bucle de eventos se define como la parte más esencial de cualquier aplicación interactiva: reaccionar a eventos permite al usuario interactuar con la aplicación [3]. Los

eventos son cosas que pueden ocurrir en un programa, como clic de ratón, movimiento de ratón, pulsación de teclado o acción de joystick [3].

El modelo es deliberadamente simple y pedagógico:

1. El sistema operativo acumula eventos en una **cola**.
2. En cada cuadro, el programa pide la lista con pygame.event.get(), que devuelve una lista de objetos Event.
3. Cada Event tiene un atributo type (QUIT, KEYDOWN, KEYUP, MOUSEMOTION, MOUSEBUTTONUP, etc.) y atributos según el tipo: event.key para teclado, event.pos para ratón.
4. El programa decide con if / elif qué hacer con cada evento.

El patrón clásico de Dodger lo muestra con claridad: en KEYDOWN se activan banderas de movimiento (moveLeft = True), en KEYUP se desactivan, y en MOUSEMOTION se mueve el rectángulo del jugador a la posición del cursor [6]. Esa separación entre "el evento ocurre" y "el estado cambia" es la lección: el evento no mueve directamente al jugador; el evento cambia una variable, y la fase de actualización usa esa variable para moverlo. Esa es la diferencia entre un programa reactivo frágil y un bucle de juego ordenado.

Los eventos también permiten discutir **eventos personalizados** y **temporizadores** (pygame.time.set_timer), que generan un evento cada cierto intervalo: aparición de enemigos, cuenta regresiva, cambio de nivel. Así, el mismo mecanismo que sirve para el teclado sirve para la lógica interna del juego.

Tipo de evento	Atributo útil	Uso típico
QUIT	—	Cerrar el juego
KEYDOWN / KEYUP	event.key, event.unicode	Mover, disparar, pausar, trucos
MOUSEMOTION	event.pos, event.rel	Seguir al cursor, arrastrar
MOUSEBUTTONDOWN / MOUSEBUTTONUP	event.pos, event.button	Clic en botones, disparar
Evento personalizado	event.type propio	Aparecer enemigos, temporizad

4.3 Estructuras de datos: el estado del juego es una estructura

Si el bucle es el corazón y los eventos son los sentidos, las estructuras de datos son la memoria del juego. Todo lo que cambia —posición, velocidad, vidas, puntuación, enemigos, balas— debe vivir en alguna estructura. Pygame hace visible esa necesidad porque, a diferencia de un motor, no guarda el estado por el programador.

Las estructuras de clase aparecen con roles concretos:

Tuplas. Un color es una tupla RGB (255, 0, 0); una posición puede ser una tupla (x, y); un rectángulo de dibujo puede ser (x, y, ancho, alto). El tutorial oficial define los colores como tuplas de rojo, verde y azul, cada componente entre 0 y 255 [3]. La inmutabilidad de la tupla aquí es una ventaja: un color constante no debería cambiar por accidente.

Listas. La colección por excelencia del juego: lista de enemigos, lista de balas, lista de partículas. En Dodger, baddies = [] empieza vacía y en cada cuadro se agregan nuevos enemigos como diccionarios; luego se recorren para moverlos, se dibujan, y se eliminan los que salen de la pantalla con baddies.remove(b) [6]. La lista modela "hay N cosas vivas ahora mismo", y su longitud cambia en tiempo real.

Diccionarios. Cada enemigo en Dodger es un diccionario con claves 'rect', 'speed' y 'surface' [6]. El diccionario agrupa atributos heterogéneos de una misma entidad bajo nombres legibles, lo que anticipa el paso a clases. Otros usos canónicos: el estado del juego como diccionario con claves 'player', 'stepCounter' y 'stars'; un diccionario de tecla a color para decodificar entrada; un diccionario de nombre de estado a función para máquinas de estados.

Diccionario como tabla de búsqueda. El tutorial oficial muestra una técnica que resume por qué las estructuras importan: en lugar de muchos if/elif para decodificar teclas, se crea key_dict = {K_k: BLACK, K_r: RED, ...} y el bucle se reduce a if event.key in key_dict: background = key_dict[event.key] [3]. Esa es la lección de diseño: reemplazar lógica ramificada por datos.

Conjuntos y colas. Aunque menos visibles, aparecen en colisiones por máscara, en conjuntos de teclas presionadas y en la propia cola de eventos, que es conceptualmente una cola FIFO.

Objetos propios de Pygame como estructuras. Rect, Surface, Sprite y Group son estructuras de datos con comportamiento. Un Sprite agrupa image y rect; un Group agrupa sprites y permite actualizarlos y dibujarlos en lote. La detección de colisiones se apoya en esas estructuras: spritecollideany() toma un Sprite y un Group, revisa cada objeto del grupo y verifica si su .rect se intersecta con el .rect del sprite; si es así, devuelve True, en caso contrario False [9]. Esa frase resume la conexión entre estructuras y juego: colisión es intersección de rectángulos, y grupo es la colección sobre la que se pregunta.

Estructura	Ejemplo en Pygame	Qué modela
Tupla	(255, 0, 0), (x, y)	Color, posición, tamaño fijo

Estructura	Ejemplo en Pygame	Qué modela
Lista	baddies, balas, particulas	Colección de entidades vivas
Diccionario	{'rect': ..., 'speed': ..., 'surface': ...}	Atributos de una entidad
Diccionario	key_dict, estados	Tabla de búsqueda, máquina de estados
Rect	playerRect	Posición y tamaño con colisión
Group	enemigos	Conjunto de sprites con actualización y dibujo en lote
Cola	pygame.event.get()	Eventos pendientes

4.4 Funciones: dividir el juego en piezas con nombre

Pygame empuja naturalmente a la descomposición en funciones porque el bucle principal, si se deja crecer, se vuelve ilegible. Los programas introductorios muestran tres funciones que aparecen una y otra vez:

- **terminate()**: apaga Pygame y sale del programa. Se llama desde varios lugares (botón de cierre, tecla Escape), lo que muestra la reutilización: una sola definición, muchas llamadas 6.
- **waitForPlayerToPressKey()**: un bucle pequeño que espera a que el jugador pulse una tecla. Encapsula "pantalla de inicio" como una idea.
- **playerHasHitBaddie(playerRect, baddies)**: recibe el rectángulo del jugador y la lista de enemigos, recorre la lista y devuelve True si algún collidererect es verdadero 6. Es el ejemplo perfecto de función con parámetros y valor de retorno: la pregunta "¿chocó?" se separa del resto del juego.
- **drawText(text, font, surface, x, y)**: recibe texto, fuente, superficie y coordenadas, renderiza el texto y lo dibuja 6. Sin esa función, cada texto requeriría repetir cuatro líneas.

Más allá de esos ejemplos, la arquitectura recomendada es funcional por fases: manejar_eventos(), actualizar(dt), dibujar(pantalla). Cada fase es una función que recibe el estado y lo transforma o lo dibuja. Esa división es la misma que se usa en motores profesionales y en el patrón Update Method: alternar entre update y draw para que cada cuadro sea diferente del anterior. En Pygame Zero, esa idea se vuelve explícita con funciones draw() y update() que el marco llama en alternancia 10.

Las funciones también permiten introducir **parámetros con valores por defecto**, **retorno múltiple** (devolver (x, y) o (colisiono, puntos)), **funciones como valores** (diccionario de estado a función) y **alcance de variables** (por qué el estado del juego suele pasarse como argumento o encapsularse en un objeto, en lugar de depender de variables globales). El juego hace visible el costo de las globales: si todo es global, cualquier función puede romper cualquier parte; si el estado se pasa explícitamente, el flujo es rastreable.

4.5 Estados del juego: el if que organiza todo el programa

Un juego real no está siempre "jugando": tiene pantalla de inicio, partida en curso, pausa y fin de partida. La forma más sencilla de modelarlo es una variable de estado, por ejemplo estado = "inicio", y un if / elif en la actualización y en el dibujo. Mientras el estado sea "jugando", se mueven las cosas; cuando es "fin", se muestra el mensaje y se espera una tecla para reiniciar. Una técnica recomendada es guardar ese estado y, en el paso de dibujo, mostrar la puntuación y un mensaje de "pulsa Espacio" cuando terminó, y reiniciar el juego al pulsar Espacio **18**.

La versión más avanzada del mismo patrón es un diccionario que mapea nombre de estado a función: estados = {"inicio": dibujar_inicio, "jugando": actualizar_juego, "fin": dibujar_fin}. Entonces el bucle se reduce a estados[estado](). Esa es otra aparición del diccionario como tabla de búsqueda: en lugar de una cadena larga de if / elif, los datos eligen la función. En proyectos grandes, cada estado puede ser una clase con sus propios métodos manejar_eventos, actualizar y dibujar, lo que anticipa la programación orientada a objetos sin necesidad de un motor.

4.6 Colisiones en detalle: de rectángulos a píxeles

La colisión es la parte del juego donde las estructuras de datos se vuelven física. Pygame ofrece tres niveles de precisión, de más rápido y aproximado a más lento y exacto **19**:

Prueba	Qué compara	Cuándo conviene
collide_rect	Si los rectángulos se solapan	Casi siempre; es la opción por defect
collide_circle	Si los círculos alrededor de los sprites se solapan	Objetos redondos, como pelotas
collide_mask	Si los píxeles visibles se solapan	Formas irregulares, naves, asteroides

Las funciones de alto nivel son:

Función	Devuelve
spritecollide(sprite, group, dokill)	Lista de sprites del grupo que tocan al sprite

Función	Devuelve
spritecollideany(sprite, group)	El primer sprite que toca, o None; la más rápida cuando solo importa s o no
groupcollide(group1, group2, dokill1, dokill2)	Diccionario: cada sprite del primer grupo que chocó, a la lista de sprite del segundo grupo con los que chocó

El parámetro dokill=True elimina los sprites que chocaron de todos sus grupos, lo que conviene para monedas y balas; con False el sprite permanece, como una pared o el jugador [19]. Un ejemplo canónico es groupcollide(lasers, enemigos, True, True), que encuentra cada láser que toca a un enemigo y elimina ambos [19].

Las colisiones por rectángulo cuentan las esquinas transparentes de una imagen, así que un enemigo redondo "choca" antes de tocar visiblemente [19]. Para eso sirve la máscara: registra qué píxeles de la imagen son sólidos y collide_mask compara esos píxeles [19]. La práctica recomendada es crear la máscara una sola vez en __init__ con self.mask = pygame.mask.from_surface(self.image), porque si no existe, collide_mask construye una nueva en cada revisión, lo que es lento [19]. Como las máscaras cuestan más que los rectángulos, un patrón común es hacer primero una revisión barata con spritecollideany y solo después la prueba de máscara sobre los sprites encontrados [19].

5. Qué significa que Pygame permite "tomar mucho o poco" de la librería

La frase apunta a tres niveles distintos de "tomar", que conviene no confundir:

5.1 Nivel 1: Tomar pocos o muchos módulos

Pygame es un conjunto de módulos, cada uno con un trabajo [5]. Nada obliga a usarlos todos. Un programa mínimo puede usar solo ventana y dibujo:

import pygamepygame.init()pantalla = pygame.display.set_mode((640, 480))

Un juego completo puede sumar sprite, mixer, font, time y transform. Esa gradualidad es pedagógica: el curso puede introducir un módulo por semana sin reescribir lo anterior. Cada módulo nuevo resuelve un problema nuevo que el estudiante ya sintió: "¿cómo pongo texto?" → font; "¿cómo suena?" → mixer; "¿cómo organizo 50 enemigos?" → sprite.

5.2 Nivel 2: Tomar poca o mucha abstracción

Dentro de un mismo problema, Pygame ofrece un espectro de abstracción:

Problema	Poco de la librería (manual)	Mucho de la librería (abstraído)
Mover enemigos	Lista de diccionarios + for + Rect.move_ip	pygame.sprite.Group + Group.update() + Group.draw()
Colisiones	rect1.colliderect(rect2) en un for	pygame.sprite.spritecollide(), groupcollide(), spritecollideany()
Entrada	pygame.event.get() + if event.type == KEYDOWN	pygame.key.get_pressed() para estado continu
Sonido	pygame.mixer.Sound('...').play()	pygame.mixer.music con bucle, volumen y colas
Texto	font.render(...) + blit manual	Función propia drawText() construida sobre fo

"Tomar poco" significa escribir más código propio y entender el mecanismo; "tomar mucho" significa delegar en la librería y avanzar más rápido. Ninguno es superior en abstracto: en clase, tomar poco al inicio enseña; tomar mucho después permite terminar un proyecto. La decisión es una decisión de diseño, no una obligación de la herramienta.

5.3 Nivel 3: Inicializar poco o mucho

pygame.init() inicializa todos los módulos importados de Pygame y devuelve una tupla de inicializaciones exitosas y fallidas **11**. Pero es posible inicializar selectivamente para velocidad: pygame.display.init() en lugar de pygame.init() completo si no se necesita audio **11**. Esa es la versión más literal de "tomar poco": si el programa no usa sonido, no se inicializa el mezclador.

También hay un nivel de importación: import pygame importa todos los submódulos disponibles de Pygame en el paquete, mientras que from pygame.locals import * es opcional y coloca un conjunto limitado de constantes y funciones en el espacio de nombres global del script, para escribir QUIT en lugar de pygame.QUIT **12**. La segunda línea ahorra escritura, pero a costa de claridad; en clase conviene preferir el prefijo pygame. hasta que el código sea largo.

5.4 Pygame frente a un motor: la diferencia que explica la frase

La comparación con Godot deja clara la idea:

Pygame is a Python library. Your program is the game: you write the loop, the drawing, the collisions and the level layout in code. Godot is a game engine with an editor. You build scenes from nodes in the editor, attach scripts, and the engine runs the loop, the physics and the drawing. ***2***

En Godot, el motor "toma" casi todo: bucle, física, dibujo. En Pygame, quien programa decide cuánto tomar. Esa es la razón por la que Pygame es mejor para aprender cómo funcionan los juegos y Godot es mejor para publicar un juego pulido [2]. Y es la razón por la que existe Pygame Zero: su objetivo declarado es permitir enseñar programación básica sin necesidad de explicar la API de Pygame ni escribir un bucle de eventos, porque las piezas comunes que en Pygame hay que cablear a mano ya vienen pre-conectadas [10]. Pygame Zero "toma mucho"; Pygame "deja elegir".

5.5 Pygame Zero: el extremo de "tomar mucho"

Pygame Zero, el paquete pgzero, es una capa encima de Pygame hecha para enseñar: escribe el bucle de juego por quien programa. Se definen una función draw() y una función update(), y Pygame Zero las llama sesenta veces por segundo; las imágenes se convierten en objetos Actor que se cargan desde una carpeta images por nombre [20]. En Pygame Zero, un juego de nave que se mueve con flechas es así:

import pgzrunWIDTH = 480HEIGHT = 360nave = Actor("ship")nave.pos = (240, 180)def draw(): screen.fill((10, 12, 30)) nave.draw()def update(): if keyboard.left: nave.x -= 5 if keyboard.right: nave.x += 5pgzrun.go()

En Pygame puro, el mismo programa escribe unas diez líneas más, y cada una es algo visible y modificable: la ventana, el reloj, el bucle de eventos y el flip() del cuadro [20]. La comparación lado a lado es clara: en Pygame Zero el bucle está oculto y se escribe draw() y update(); en Pygame se escribe el bucle while; las imágenes son Actor("ship") frente a pygame.image.load() y blit(); la entrada es keyboard.left frente a pygame.key.get_pressed() y eventos; el sonido es sounds.coin.play() frente a pygame.mixer.Sound("coin.ogg").play(); y los temporizadores son clock.schedule(fn, 2.0) frente a pygame.time.set_timer() o get_ticks() [20].

Pygame Zero es bueno para una primera tarde, un aprendiz joven o una clase, porque un personaje en movimiento requiere muy pocas líneas; pero sus convenciones, carpetas images y sounds y nombres globales mágicos, son propias, y el código de un tutorial de Pygame Zero no corre en Pygame puro [20]. Pygame puro es el mejor lugar para aprender si se quiere entender los juegos, porque el bucle, los eventos y el dibujo están en el código, y nada queda oculto cuando un juego más grande necesita control [20]. El paso de uno a otro no es grande: un Actor se convierte en imagen más Rect, draw() se convierte en la parte de dibujo del bucle y update() en su parte de actualización [20].

6. Ruta de proyectos: de un cuadrado a un juego completo

Un buen primer proyecto es un juego con una regla y un control: atrapar objetos que caen, Pong o Snake. Cada uno cabe en menos de 100 líneas y usa todo el bucle, entrada, actualización, colisiones, dibujo y puntuación, sin necesitar imágenes [18]. La ruta progresiva recomendada es:

Dos detalles de estos proyectos ilustran estructuras de datos en acción. En Snake, el cuerpo es una lista de celdas: cada paso agrega una cabeza nueva y quita la cola, salvo que la serpiente acabe de comer [18]. En el juego de nave que vuela por una cueva, drone.collidelist(rocks) devuelve el índice de la roca que toca el dron, o -1 cuando no toca ninguna, de modo que una línea revisa todas las rocas [18]. En Pong, cada golpe acelera la pelota un 5%, lo que muestra cómo una variable de velocidad puede cambiar con las reglas [18].

Para mejorar cada proyecto sin reescribirlo, las extensiones naturales son: cambiar las formas por imágenes con image.load y blit, agregar efectos de sonido con pygame.mixer, y convertir los objetos en sprites cuando hay muchos tipos **18**. Esa es la misma idea de gradualidad del informe: el proyecto crece tomando un poco más de la librería en cada etapa.

7. Errores comunes y cómo depurarlos

La mayoría de los errores de Pygame provienen de un puñado de equivocaciones **21**. La siguiente tabla funciona como lista de revisión antes de pedir ayuda:

Cuando un programa se comporta mal y no se ve por qué, conviene compararlo con una plantilla mínima que tenga cada pieza en el orden correcto: eventos, actualización, fill, dibujo, flip, tick, y pygame.quit() solo después del bucle **21**. Esa plantilla es también una buena forma de estudiar: si la plantilla funciona y el juego no, la diferencia está en lo que el juego agregó.

8. Ejercicios propuestos para clase

1. **Cambiar un número y observar.** Abrir un juego que funcione, cambiar la velocidad, el tamaño de la ventana o el color, y describir qué cambió. Es la forma más rápida de conectar código con resultado visible.
2. **Agregar una regla.** En el juego de atrapar monedas, agregar que las monedas doradas valen doble. Obliga a distinguir tipos de entidades con un campo más en el diccionario o con un grupo distinto.
3. **Agregar un estado.** Convertir un juego sin fin en un juego con pantalla de inicio y fin de partida usando una variable estado.
4. **Pasar de lista a sprites.** Reescribir un juego de lista de diccionarios usando Sprite y Group, y comparar cuántas líneas desaparecen del bucle principal.
5. **Cambiar la precisión de colisión.** Pasar de collide_rect a collide_mask en un juego con imágenes irregulares y observar si el juego se siente más justo y si baja la velocidad.
6. **Medir con delta time.** Cambiar el movimiento de píxeles por cuadro a velocidad por segundo con dt = clock.tick(60) / 1000 y comprobar que el juego corre igual a 30 y a 60 FPS.
7. **Documentar el bucle.** En un programa ajeno, marcar con comentarios las cuatro fases, eventos, actualización, dibujo y tiempo, y señalar dónde vive cada estructura de datos.

9. Ejemplo integrador: todo junto en un programa mínimo

El siguiente esqueleto muestra, en un solo lugar, los cuatro temas de clase y la idea de "tomar poco":

import pygame# Constantes y estructuras: tuplas para colores, diccionario para teclasBLANCO = (255, 255, 255)ROJO = (255, 0, 0)TECLAS_COLOR = {pygame.K_r: ROJO, pygame.K_b: (0, 0, 255)}def dibujar_texto(texto, fuente, superficie, x, y): imagen = fuente.render(texto, True, BLANCO) superficie.blit(imagen, (x, y))def main(): pygame.init() pantalla = pygame.display.set_mode((640, 480)) reloj = pygame.time.Clock() fuente = pygame.font.SysFont(None, 36) jugador = pygame.Rect(300, 200, 50, 50) # estructura Rect color = ROJO corriendo = True while

corriendo: # ciclo principal for evento in pygame.event.get(): # eventos if evento.type == pygame.QUIT: corriendo = False if evento.type == pygame.KEYDOWN and evento.key in TECLAS_COLOR: color = TECLAS_COLOR[evento.key] teclas = pygame.key.get_pressed() if teclas[pygame.K_LEFT]: jugador.x -= 5 if teclas[pygame.K_RIGHT]: jugador.x += 5 pantalla.fill((0, 0, 0)) pygame.draw.rect(pantalla, color, jugador) dibujar_texto("Usa flechas y R/B", fuente, pantalla, 10, 10) pygame.display.flip() reloj.tick(60) pygame.quit()main()

En ese programa, el ciclo es el juego, el evento cambia el estado, la tupla y el diccionario guardan datos, el Rect guarda posición, y la función dibujar_texto evita repetir código. Si se quisiera "tomar más" de la librería, el jugador se convertiría en Sprite, los enemigos en Group, y la colisión en spritecollideany(); la lógica sería la misma, solo cambiaría el nivel de abstracción.

10. Limitaciones y buenas prácticas para clase

- **No confundir biblioteca con motor.** Pygame no trae física, animación por huesos, editor de niveles ni publicación a móvil. Si el proyecto necesita eso, el paso siguiente es un motor como Godot, no más Pygame.
- **Nombrar archivos con cuidado.** Nunca llamar al script pygame.py, porque Python importará ese archivo en lugar de la biblioteca y producirá un error de inicialización parcial [11].
- **Usar pygame. como prefijo al inicio.** Evitar from pygame.locals import * hasta que el código sea lo bastante largo para justificarlo; la claridad vale más que ahorrar escritura.
- **Cerrar siempre con pygame.quit().** El bucle debe tener una salida ordenada, no depender de cerrar la consola.
- **Separar actualización de dibujo.** Si en el mismo bloque se mueve y se dibuja, el programa se vuelve difícil de depurar. La separación en fases —eventos, actualización, dibujo— es la disciplina central del curso.
- **Elegir conscientemente el nivel de abstracción.** Empezar con listas y Rect para entender colisiones; pasar a Sprite y Group cuando el número de entidades lo justifique.

Conclusión

Pygame es una biblioteca de módulos sobre SDL para crear aplicaciones multimedia y videojuegos 2D en Python, cuya principal virtud pedagógica es que no oculta el funcionamiento del juego. Los ciclos se materializan en el game loop; los eventos se materializan en la cola que se recorre en cada cuadro; las estructuras de datos se materializan en el estado del juego —tuplas para colores, listas para entidades, diccionarios para atributos y tablas de búsqueda, Rect y Group para posición y colecciones—; y las funciones se materializan en la descomposición del juego en piezas con nombre. Que Pygame permita "tomar mucho o poco" significa que quien programa elige cuántos módulos usar, cuánta abstracción delegar en la librería y cuántos submódulos inicializar, desde un experimento visual de veinte líneas hasta un juego completo con sprites, sonido, texto y temporizadores. Esa gradualidad es lo que lo convierte en una herramienta adecuada para un curso que avanza de los fundamentos de Python a un proyecto interactivo completo.

Sources

1 PyPI — pygame
2 Coddy — Pygame vs Godot: Which Should You Learn?
3 Pygame Tutorial — Introduction to Pygame
4 Wikipedia — Pygame
5 Coddy — What Is Pygame? The Python Game Library Explained
6 Invent with Python — IYOCGwP, Chapter 20 - Dodger
7 Berea College — 17. PyGame — How to Think Like a Computer Scientist: Learning with Python 3
8 Coddy — Pygame Clock: tick(), FPS and Delta Time
9 Real Python — Collision Detection (Video) – Real Python
10 Matt Layman — Teaching a kid to code with Pygame Zero
11 CopyProgramming — Fixing pygame.init Error: Duplicate Issues and 2026 Best Practices
12 ExchangeTuts — Can someone please explain this weird Pygame importing convention?
13 Pygame-ce Docs — Pygame Intro
14 Wikipedia — Pygame
15 Coddy — How to Install Pygame with pip (pygame-ce)
16 CopyProgramming — Understanding Pygame Rectangle Drawing Direction in Python
17 Coddy — Pygame Sprite and Group Classes Explained
18 Coddy — Pygame Examples: Small Games You Can Run and Edit
19 Coddy — Pygame Sprite Collisions: spritecollide and Masks
20 Coddy — Pygame Zero vs Pygame: What Is the Difference?
21 Coddy — Common Pygame Errors and How to Fix Them — Can someone please explain this weird Pygame importing convention?